COMSATS UNIVERSITY ISLAMABAD
SAHIWAL CAMPUS

Department Computer Science

ASSIGNMENT 4

5/17/2026

SUBMITTED BY:

Zarmeen Rasool

SUBMITTED TO:

Sir.InaamulHaq

REGISTRATION NUMBER:

FA23-BCS-112

SECTION:

6th (B)

Topic:

Transactions using MongoDB

DATE OF SUBMISSION:

May 18, 2026

Table of Contents

ASSIGNMENT 4

Question:

- **Create a MongoDB database named BankDB and a collection named accounts. Insert sample account records containing the account holder name, balance, and version number.**
- **Write MongoDB queries and Python code to perform basic CRUD operations on the accounts collection.**
- **Simulate concurrent updates on the same account without using concurrency control and explain the resulting race condition or lost update problem.**
- **Implement Optimistic Concurrency Control (OCC) in MongoDB using a version field and demonstrate how conflicting updates are detected.**
- **Develop a transaction-based money transfer system between two accounts using MongoDB transactions and explain how ACID properties are maintained.**
- **Demonstrate transaction rollback in case of insufficient balance or system failure.**
- **Analyze how MongoDB handles consistency, isolation, and concurrent access in distributed systems.**
- **Compare normal updates with transaction-based updates in MongoDB and discuss their advantages and limitations.**
- **Explain the role of versioning, atomic operations, and transactions in concurrency control. Provide source code, execution screenshots, output results, and a brief report discussing implementation steps, observations, challenges, and conclusions.**

Solution:

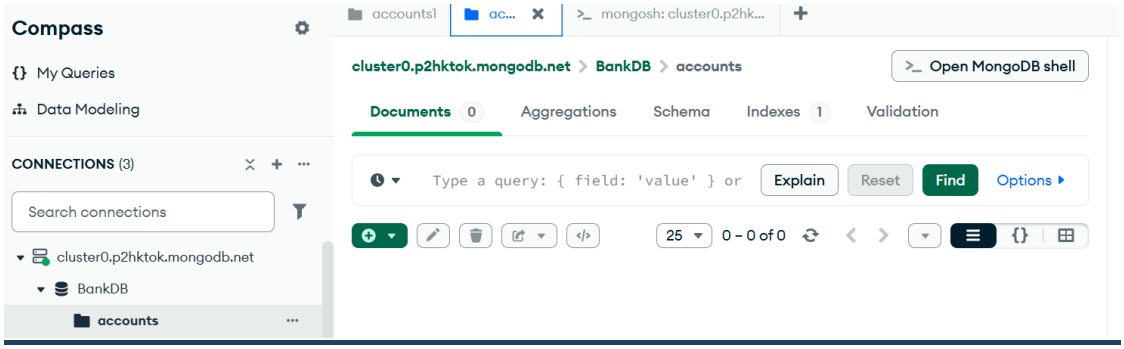
1. Database And Collection Creation:

A database is a collection of related data.

MongoDB is a NoSQL document database that stores data in BSON documents.

In this assignment:

- Database Name = BankDB
- Collection Name = accounts



1.1 Insert sample account records:

```
_MONGOSH
> use BankDB
< switched to db BankDB
> db["accounts"].find()
<
> db.accounts.insertMany([
{
    account_no:1001,
    holder_name:"Ali Khan",
    balance:50000,
    version:1
},
{
    account_no:1002,
    holder_name:"Sara Ahmed",
    balance:30000,
    version:1
},
{
    account_no:1003,
    holder_name:"Usman Tariq",
    balance:45000,
    version:1
}
```

```
  {
      account_no:1003,
      holder_name:"Usman Tariq",
      balance:45000,
      version:1
  }
  ])
< {
    acknowledged: true,
    insertedIds: {
      '0': ObjectId('6a09ed9185a3d9ff5fec6a3f'),
      '1': ObjectId('6a09ed9185a3d9ff5fec6a40'),
      '2': ObjectId('6a09ed9185a3d9ff5fec6a41')
    }
  }
Atlas atlas-i5oclg-shard-0 [primary] BankDB>
```

1.2Verify Records Enter Successfully:

```
> db.accounts.find().pretty()
< {
    _id: ObjectId('6a09ed9185a3d9ff5fec6a3f'),
    account_no: 1001,
    holder_name: 'Ali Khan',
    balance: 50000,
    version: 1
  }
  {
    _id: ObjectId('6a09ed9185a3d9ff5fec6a40'),
    account_no: 1002,
    holder_name: 'Sara Ahmed',
    balance: 30000,
    version: 1
  }
  {
    _id: ObjectId('6a09ed9185a3d9ff5fec6a41'),
    account_no: 1003,
    holder_name: 'Usman Tariq',
    balance: 45000,
    version: 1
  }
```

```
  }
  {
    _id: ObjectId('6a09ed9185a3d9ff5fec6a41'),
    account_no: 1003,
    holder_name: 'Usman Tariq',
    balance: 45000,
    version: 1
  }
Atlas atlas-i5oclg-shard-0 [primary] BankDB>
```

1.3Python Code to create databse and collection:

bankdb.py

```python
from pymongo import MongoClient

client = MongoClient("mongodb://localhost:27017/")

db = client["BankDB"]

accounts = db["accounts"]

data = [
    {
        "account_no":1001,
        "holder_name":"Ali Khan",
        "balance":50000,
        "version":1
    },
    {
        "account_no":1002,
        "holder_name":"Sara Ahmed",
        "balance":30000,
        "version":1
    },
    {
        "account_no":1003,
        "holder_name":"Usman Tariq",
```

```
    {
        "account_no":1002,
        "holder_name":"Sara Ahmed",
        "balance":30000,
        "version":1
    },
    {
        "account_no":1003,
        "holder_name":"Usman Tariq",
        "balance":45000,
        "version":1
    }
]

accounts.insert_many(data)

print("Database and Collection Created")
```

1.3.1 Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS E:\6thSemester\AdvDataBase\Python> python -u "e:\6thSemester\AdvDataBase\Python\bankdb.py"
Database and Collection Created
PS E:\6thSemester\AdvDataBase\Python>
```

2. MongoDB queries and Python code to perform basic CRUD operations:

2.1Insert Query in MongoDb:

```
> db.accounts.insertOne({
    account_no:1004,
    holder_name:"Hina Malik",
    balance:25000,
    version:1
  })
< {
    acknowledged: true,
    insertedId: ObjectId('6a09f08785a3d9ff5fec6a42')
  }
Atlas atlas-i5oclg-shard-0 [primary] BankDB>
```

2.2Read Query in MongoDb:

```
> db.accounts.find()
< {
    _id: ObjectId('6a09ed9185a3d9ff5fec6a3f'),
    account_no: 1001,
    holder_name: 'Ali Khan',
    balance: 50000,
    version: 1
  }
  {
    _id: ObjectId('6a09ed9185a3d9ff5fec6a40'),
    account_no: 1002,
    holder_name: 'Sara Ahmed',
    balance: 30000,
    version: 1
  }
  {
    _id: ObjectId('6a09ed9185a3d9ff5fec6a41'),
    account_no: 1003,
    holder_name: 'Usman Tariq',
    balance: 45000,
    version: 1
  }
```

```
  {
    _id: ObjectId('6a09ed9185a3d9ff5fec6a41'),
    account_no: 1003,
    holder_name: 'Usman Tariq',
    balance: 45000,
    version: 1
  }
  {
    _id: ObjectId('6a09f08785a3d9ff5fec6a42'),
    account_no: 1004,
    holder_name: 'Hina Malik',
    balance: 25000,
    version: 1
  }
Atlas atlas-i5oclg-shard-0 [primary] BankDB>
```

2.3Update Query in MongoDb:

2.3.1 Before Updation:

```
{
  _id: ObjectId('6a09ed9185a3d9ff5fec6a3f'),
  account_no: 1001,
  holder_name: 'Ali Khan',
  balance: 50000,
  version: 1
}
```

2.3.2 Implementation Query:

```
> db.accounts.updateOne(
    {account_no:1001},
    {$set:{balance:55000}}
  )
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
Atlas atlas-i5oclg-shard-0 [primary] BankDB>
```

2.3.3 After Updation:

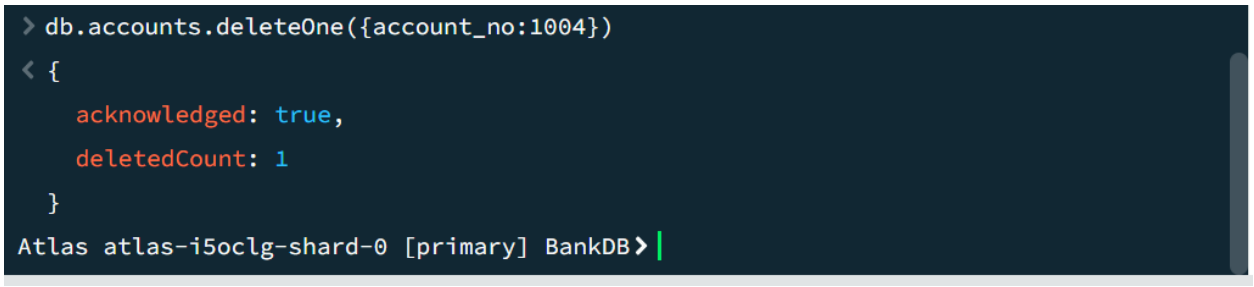
```
_id: ObjectId('6a09ed9185a3d9ff5fec6a3f')
account_no : 1001
holder_name : "Ali Khan"
balance : 55000
version : 1
```

2.4Delete Query in MongoDb:

2.4.1 Before Deletion Records:

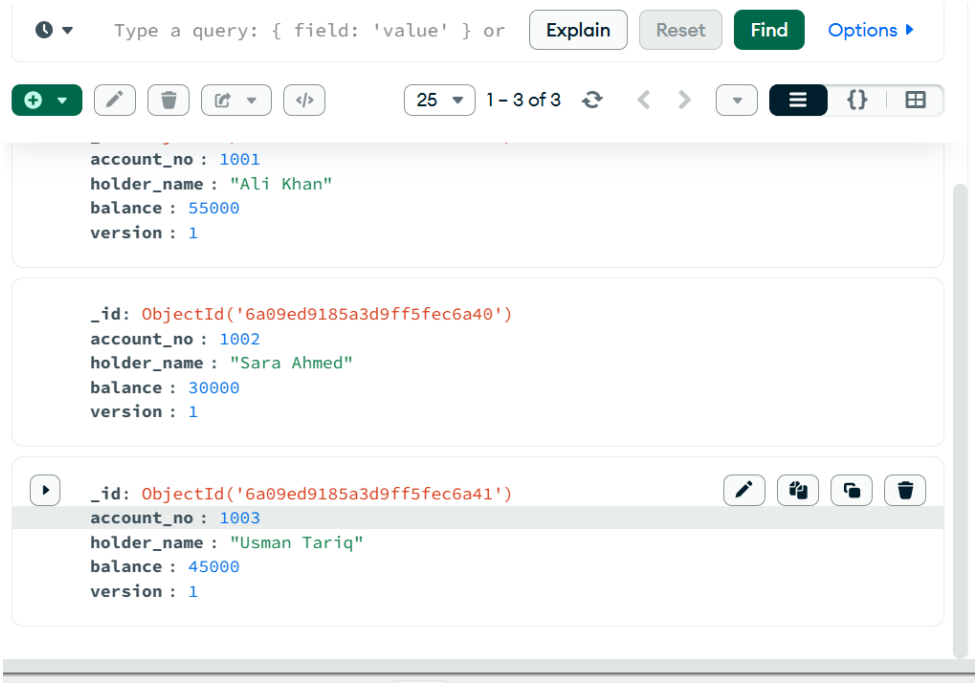
```
_id: ObjectId('6a09ed9185a3d9ff5fec6a41')
account_no : 1003
holder_name : "Usman Tariq"
balance : 45000
version : 1

_id: ObjectId('6a09f08785a3d9ff5fec6a42')
account_no : 1004
holder_name : "Hina Malik"
balance : 25000
version : 1
```

2.4.2 Implementation of Deletion:



```
> db.accounts.deleteOne({account_no:1004})
< {
    acknowledged: true,
    deletedCount: 1
  }
Atlas atlas-i5oclg-shard-0 [primary] BankDB>
```

2.4.3 After Deletion:



2.5Python Crud Operations:

bankdb.py crud.py

crud.py > ...

```python
from pymongo import MongoClient

client = MongoClient("mongodb://localhost:27017/")

db = client["BankDB"]

accounts = db["accounts"]

# INSERT

accounts.insert_one({
    "account_no":1005,
    "holder_name":"Ayesha Noor",
    "balance":40000,
    "version":1
})

print("Record Inserted")

# READ

print("\nAll Accounts")

for acc in accounts.find():
    print(acc)
```

```python
# READ

print("\nAll Accounts")

for acc in accounts.find():
    print(acc)

# UPDATE

accounts.update_one(
    {"account_no":1005},
    {"$set":{"balance":45000}}
)

print("\nBalance Updated")

# DELETE

accounts.delete_one({"account_no":1005})

print("\nRecord Deleted")
```

2.5.1 Output:

```
PS E:\6thSemester\AdvDataBase\Python> python -u "e:\6thSemester\AdvDataBase\Python\crud.py"
Record Inserted

All Accounts
{'_id': ObjectId('6a09ef94aac5a0bf8f9e737a'), 'account_no': 1001, 'holder_name': 'Ali Khan', 'balance': 50000, 'version': 1}
{'_id': ObjectId('6a09ef94aac5a0bf8f9e737b'), 'account_no': 1002, 'holder_name': 'Sara Ahmed', 'balance': 30000, 'version': 1}
{'_id': ObjectId('6a09ef94aac5a0bf8f9e737c'), 'account_no': 1003, 'holder_name': 'Usman Tariq', 'balance': 45000, 'version': 1}
{'_id': ObjectId('6a09f3521044bdc8d154d26d'), 'account_no': 1005, 'holder_name': 'Ayesha Noor', 'balance': 40000, 'version': 1}

Balance Updated

Record Deleted
PS E:\6thSemester\AdvDataBase\Python>
```

3. Concurrent updates on same account without using concurrency control:

Concurrency means multiple users accessing same data simultaneously.

Problem:

- Two users update same account together
- One update overwrites another

Called:

- Race Condition
- Lost Update Problem

Example

- Initial Balance:50000
- User AWithdraws:10000
- New balance:40000
- User B Deposits:5000
- New balance:55000
- Expected Final Balance:45000

But due to concurrency issue:

- Final balance may become: 40000 OR 55000

3.1MongoDb Shell Simulation:

3.1.1 User A Reads Data:

```
> db.accounts.findOne({account_no:1001})
< {
    _id: ObjectId('6a09ed9185a3d9ff5fec6a3f'),
    account_no: 1001,
    holder_name: 'Ali Khan',
    balance: 55000,
    version: 1
  }
Atlas atlas-i5oclg-shard-0 [primary] BankDB>
```

3.1.2 User A Update:

```
_id: ObjectId('6a09ed9185a3d9ff5fec6a3f')
account_no : 1001
holder_name : "Ali Khan"
balance : 55000
version : 1
```

```
> db.accounts.updateOne(
    {account_no:1001},
    {$set:{balance:40000}}
  )
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
```

```
_id: ObjectId('6a09ed9185a3d9ff5fec6a3f')
account_no : 1001
holder_name : "Ali Khan"
balance : 40000
version : 1
```

3.1.3 User B Update:

```
> db.accounts.updateOne(
    {account_no:1001},
    {$set:{balance:55000}}
  )
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
Atlas atlas-i5oclg-shard-0 [primary] BankDB>
```

```
_id: ObjectId('6a09ed9185a3d9ff5fec6a3f')
account_no : 1001
holder_name : "Ali Khan"
balance : 55000
version : 1
```

```
_id: ObjectId('6a09ed9185a3d9ff5fec6a40')
account_no : 1002
holder_name : "Sara Ahmed"
balance : 30000
version : 1
```

3.1.4 Final Record

```
> db.accounts.findOne({account_no:1001})
< {
    _id: ObjectId('6a09ed9185a3d9ff5fec6a3f'),
    account_no: 1001,
    holder_name: 'Ali Khan',
    balance: 55000,
    version: 1
  }
Atlas atlas-i5oclg-shard-0 [primary] BankDB>
```

3.2Python Concurrency Program:

bankdb.py crud.py concurrent.py

concurrent.py > ...

```python
from pymongo import MongoClient
import threading
import time

client = MongoClient("mongodb://localhost:27017/")

db = client["BankDB"]

accounts = db["accounts"]

def withdraw():

    acc = accounts.find_one({"account_no":1001})

    balance = acc["balance"]

    time.sleep(2)

    balance -= 10000

    accounts.update_one(
        {"account_no":1001},
        {"$set":{"balance":balance}}
    )

    print("Withdraw Completed")

def deposit():
```

```python
concurrent.py > withdraw
def withdraw():

    print("Withdraw Completed")

def deposit():

    acc = accounts.find_one({"account_no":1001})

    balance = acc["balance"]

    time.sleep(1)

    balance += 5000

    accounts.update_one(
        {"account_no":1001},
        {"$set":{"balance":balance}}
    )

    print("Deposit Completed")

t1 = threading.Thread(target=withdraw)
t2 = threading.Thread(target=deposit)

t1.start()
t2.start()

t1.join()
t2.join()
```

```python
    )

    print("Deposit Completed")

t1 = threading.Thread(target=withdraw)
t2 = threading.Thread(target=deposit)

t1.start()
t2.start()

t1.join()
t2.join()

final = accounts.find_one({"account_no":1001})

print("\nFinal Balance:", final["balance"])
```

3.2.1 Output:

```
python -u "e:\6thSemester\AdvDataBase\Python\concurrency_demo.py"
Deposit Completed
Withdraw Completed

Final Balance: 40000
PS E:\6thSemester\AdvDataBase\Python>
```

4. Implement Optimistic Concurrency Control (OCC) in MongoDB:

OCC uses a version field.

Steps:

- Read current version
- Update only if version matches
- Increment version after update

If another transaction already changed data:

- Update fails
- Conflict detected

4.1MongoDb Shell Occ

4.1.1 Reset Record

```
> db.accounts.updateOne(
  {
     account_no:1001
  },
  {
     $set:{
        balance:50000,
        version:1
     }
  }
  )
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
Atlas atlas-i5oclg-shard-0 [primary] BankDB>
```

```
_id: ObjectId('6a09ed9185a3d9ff5fec6a3f')
account_no : 1001
holder_name : "Ali Khan"
balance : 50000
version : 1
```

4.1.2 OCC Update:

```
> db.accounts.updateOne(
  {
     account_no:1001,
     version:1
  },
  {
     $set:{balance:45000},
     $inc:{version:1}
  }
  )
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
Atlas atlas-i5oclg-shard-0 [primary] BankDB>
```

```
_id: ObjectId('6a09ed9185a3d9ff5fec6a3f')
account_no : 1001
holder_name : "Ali Khan"
balance : 45000
version : 2
```

4.1.3 Conflict Update:

```
> db.accounts.updateOne(
  {
     account_no:1001,
     version:1
  },
  {
     $set:{balance:55000},
     $inc:{version:1}
  }
  )
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 0,
    modifiedCount: 0,
    upsertedCount: 0
  }
Atlas atlas-i5oclg-shard-0 [primary] BankDB>
```

```
_id: ObjectId('6a09ed9185a3d9ff5fec6a3f')
account_no : 1001
holder_name : "Ali Khan"
balance : 45000
version : 2
```

4.1.4 Conflict Result:

modifiedCount:0

- Conflict detected
- Record already modified

4.2Python Occ Program:

bankdb.py crud.py concurrency_demo.py occ.py

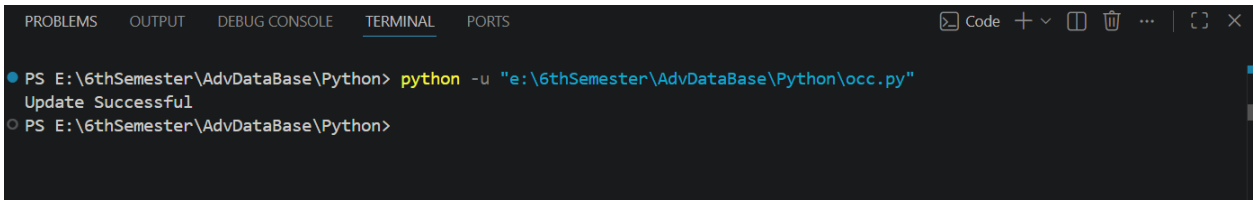
occ.py > ...

```python
from pymongo import MongoClient

client = MongoClient("mongodb://localhost:27017/")

db = client["BankDB"]

accounts = db["accounts"]

acc = accounts.find_one({"account_no":1001})

current_balance = acc["balance"]

current_version = acc["version"]

new_balance = current_balance - 5000

result = accounts.update_one(
    {
        "account_no":1001,
        "version":current_version
    },
    {
        "$set":{"balance":new_balance},
        "$inc":{"version":1}
    }
)

if result.modified_count == 1:
    print("Update Successful")
```

occ.py > ...

```python
new_balance = current_balance - 5000

result = accounts.update_one(
    {
        "account_no":1001,
        "vDersion":current_version
    },
    {
        "$set":{"balance":new_balance},
        "$inc":{"version":1}
    }
)

if result.modified_count == 1:
    print("Update Successful")
else:
    print("Conflict Detected")
```

4.2.1 Output:



5. Transaction-based MongoDb and explain how ACID properties are maintained:

A transaction is a group of operations executed as one unit.

MongoDB transactions maintain ACID properties.

5.1 Acid Properties:

Property	Meaning
Atomicity	All operations succeed or fail together
Consistency	Database remains valid
Isolation	Transactions do not interfere
Durability	Data remains permanent

5.2MongoDb Shell Transaction:

5.2.1 Replica Set:

```
> rs.status()
< {
    set: 'atlas-i5oclg-shard-0',
    date: 2026-05-17T17:45:13.499Z,
    myState: 1,
    term: Long('285'),
    syncSourceHost: '',
    syncSourceId: -1,
    heartbeatIntervalMillis: Long('2000'),
    majorityVoteCount: 2,
    writeMajorityCount: 2,
    votingMembersCount: 3,
    writableVotingMembersCount: 3,
    optimes: {
      lastCommittedOpTime: { ts: Timestamp({ t: 1779039913, i: 4 }), t: Long('285')
      lastCommittedWallTime: 2026-05-17T17:45:13.367Z,
      readConcernMajorityOpTime: { ts: Timestamp({ t: 1779039913, i: 4 }), t: Long(
      appliedOpTime: { ts: Timestamp({ t: 1779039913, i: 4 }), t: Long('285') },
      durableOpTime: { ts: Timestamp({ t: 1779039913, i: 4 }), t: Long('285') },
      writtenOpTime: { ts: Timestamp({ t: 1779039913, i: 4 }), t: Long('285') },
      lastAppliedWallTime: 2026-05-17T17:45:13.367Z,
      lastDurableWallTime: 2026-05-17T17:45:13.367Z,
```

```
},
members: [
  {
    _id: 0,
    name: 'ac-tfd1u4t-shard-00-00.p2hktok.mongodb.net:27017',
    health: 1,
    state: 2,
    stateStr: 'SECONDARY',
    uptime: 789807,
    optime: { ts: Timestamp({ t: 1779039912, i: 2 }), t: Long('285') },
    optimeDurable: { ts: Timestamp({ t: 1779039912, i: 2 }), t: Long('285') },
    optimeWritten: { ts: Timestamp({ t: 1779039912, i: 2 }), t: Long('285') },
    optimeDate: 2026-05-17T17:45:12.000Z,
    optimeDurableDate: 2026-05-17T17:45:12.000Z,
    optimeWrittenDate: 2026-05-17T17:45:12.000Z,
    lastAppliedWallTime: 2026-05-17T17:45:13.367Z,
    lastDurableWallTime: 2026-05-17T17:45:13.367Z,
    lastWrittenWallTime: 2026-05-17T17:45:13.367Z,
    lastHeartbeat: 2026-05-17T17:45:12.229Z,
    lastHeartbeatRecv: 2026-05-17T17:45:13.046Z,
    pingMs: Long('0'),
    lastHeartbeatMessage: '',
```

```
    _id: 1,
    name: 'ac-tfd1u4t-shard-00-01.p2hktok.mongodb.net:27017',
    health: 1,
    state: 1,
    stateStr: 'PRIMARY',
    uptime: 789847,
    optime: { ts: Timestamp({ t: 1779039913, i: 4 }), t: Long('285') },
    optimeDate: 2026-05-17T17:45:13.000Z,
    optimeWritten: { ts: Timestamp({ t: 1779039913, i: 4 }), t: Long('285') },
    optimeWrittenDate: 2026-05-17T17:45:13.000Z,
    lastAppliedWallTime: 2026-05-17T17:45:13.367Z,
    lastDurableWallTime: 2026-05-17T17:45:13.367Z,
    lastWrittenWallTime: 2026-05-17T17:45:13.367Z,
    syncSourceHost: '',
    syncSourceId: -1,
    infoMessage: '',
    electionTime: Timestamp({ t: 1778250372, i: 1 }),
    electionDate: 2026-05-08T14:26:12.000Z,
    configVersion: 104,
    configTerm: 285,
    self: true,
    lastHeartbeatMessage: ''
```

```
  ],
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1779039913, i: 4 }),
    signature: {
      hash: Binary.createFromBase64('BpbM92o12aZW5DAlpHe02I5CRu4=', 0),
      keyId: Long('7610459603565805579')
    }
  },
  operationTime: Timestamp({ t: 1779039913, i: 4 })
}
tlas atlas-i5oclg-shard-0 [primary] BankDB>
```

5.2.2 Start Session:

```
> session = db.getMongo().startSession()
< {
    id: UUID('47c7cdfb-6ba8-4374-8dce-0415438ee0d5')
  }
Atlas atlas-i5oclg-shard-0 [primary] BankDB>
```

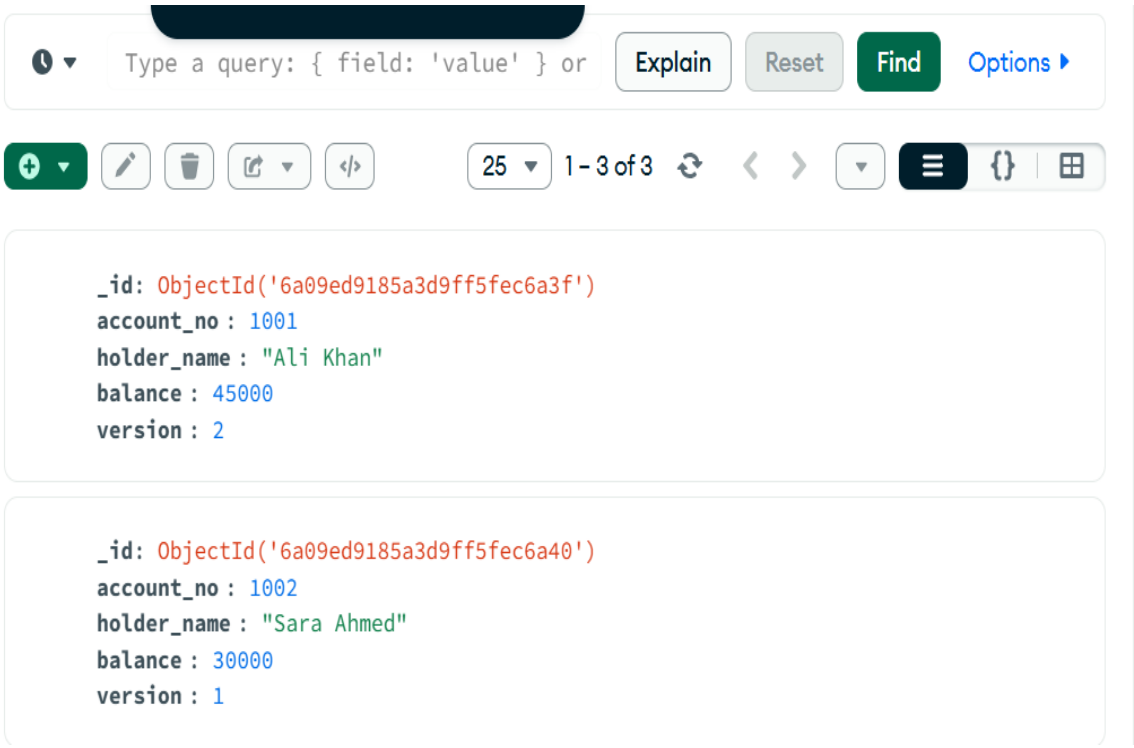
5.2.3 Select Database:

```
> sessionDB = session.getDatabase("BankDB")
< BankDB
Atlas atlas-i5oclg-shard-0 [primary] BankDB>
```

5.2.4 Start Transaction:

```
> session.startTransaction()
Atlas atlas-i5oclg-shard-0 [primary] BankDB>
```

5.2.5 Deduct Money:




```
> session.startTransaction()
> sessionDB.accounts.updateOne(
    {account_no:1001},
    {$inc:{balance:-5000}}
  )
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
Atlas atlas-i5oclg-shard-0 [primary] BankDB>
```

5.2.6 Add Money:

```
> sessionDB.accounts.updateOne(
    {account_no:1002},
    {$inc:{balance:5000}}
  )
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
Atlas atlas-i5oclg-shard-0 [primary] BankDB>
```

5.2.7 Commit Transaction:

```
  }
> session.commitTransaction()
Atlas atlas-i5oclg-shard-0 [primary] BankDB>
```

5.2.8 Verify Records:

```
_id: ObjectId('6a09ed9185a3d9ff5fec6a3f')
account_no : 1001
holder_name : "Ali Khan"
balance : 40000
version : 2
```

```
_id: ObjectId('6a09ed9185a3d9ff5fec6a40')
account_no : 1002
holder_name : "Sara Ahmed"
balance : 35000
version : 1
```

5.2.9 Session End:

```
> session.commitTransaction()
> session.endSession()
Atlas atlas-i5oclg-shard-0 [primary] BankDB>
```

5.3Python Transaction Program:

transaction.py > ...

```python
from pymongo import MongoClient

client = MongoClient("mongodb://localhost:27017/")

db = client["BankDB"]

accounts = db["accounts"]

session = client.start_session()

try:

    with session.start_transaction():

        sender = accounts.find_one(
            {"account_no":1001},
            session=session
        )

        if sender["balance"] < 5000:
            raise Exception("Insufficient Balance")

        accounts.update_one(
            {"account_no":1001},
            {"$inc":{"balance":-5000}},
            session=session
        )

        accounts.update_one(
```

```python
            if sender["balance"] < 5000:
                raise Exception("Insufficient Balance")

            accounts.update_one(
                {"account_no":1001},
                {"$inc":{"balance":-5000}},
                session=session
            )

            accounts.update_one(
                {"account_no":1002},
                {"$inc":{"balance":5000}},
                session=session
            )

            print("Transaction Successful")

    except Exception as e:

        print("Transaction Failed:", e)

    finally:

        session.end_session()
```

6. Transaction rollback in case of insufficient balance or system failure:

Rollback means undoing all operations if transaction fails.

Example:

- Insufficient balance
- System crash
- Network failure

No partial update occurs.

6.1 MongoDb Shell Rollback:

6.1.1 Start Transaction:

```
> session = db.getMongo().startSession()
< {
    id: UUID('47c7cdfb-6ba8-4374-8dce-0415438ee0d5')
  }
Atlas atlas-i5oclg-shard-0 [primary] BankDB>
```

6.1.2 Session DB:

```
  }
> sessionDB = session.getDatabase("BankDB")
< BankDB
Atlas atlas-i5oclg-shard-0 [primary] BankDB>
```

6.1.3 Start Transaction:

```
> sessionDB = session.getDatabase("BankDB")
< BankDB
> session.startTransaction()
Atlas atlas-i5oclg-shard-0 [primary] BankDB>
```

6.1.4 Read Sender:

```
sender = sessionDB.accounts.findOne(
   {account_no:1001}
)
{
  _id: ObjectId('6a09ed9185a3d9ff5fec6a3f'),
  account_no: 1001,
  holder_name: 'Ali Khan',
  balance: 40000,
  version: 2
}
tlas atlas-i5oclg-shard-0 [primary] BankDB>
```

6.1.5 Check Balance:

```
if(sender.balance < 100000)
 {
    session.abortTransaction()

    print("Transaction Aborted - Insufficient Balance")
 }
Transaction Aborted - Insufficient Balance
Atlas atlas-i5oclg-shard-0 [primary] BankDB>
```

6.1.6 Verify No Changes Occurred:

```
> db.accounts.find().pretty()
< {
    _id: ObjectId('6a09ed9185a3d9ff5fec6a3f'),
    account_no: 1001,
    holder_name: 'Ali Khan',
    balance: 40000,
    version: 2
  }
  {
    _id: ObjectId('6a09ed9185a3d9ff5fec6a40'),
    account_no: 1002,
    holder_name: 'Sara Ahmed',
    balance: 35000,
    version: 1
  }
  {
    _id: ObjectId('6a09ed9185a3d9ff5fec6a41'),
    account_no: 1003,
    holder_name: 'Usman Tariq',
    balance: 45000,
    version: 1
  }
```

6.2Python Rollback:

transaction.py > ...

```python
from pymongo import MongoClient

client = MongoClient("mongodb://localhost:27017/")

db = client["BankDB"]

accounts = db["accounts"]

session = client.start_session()

try:

    with session.start_transaction():

        sender = accounts.find_one(
            {"account_no":1001},
            session=session
        )

        if sender["balance"] < 10000:
            raise Exception("Insufficient Balance")

        accounts.update_one(
            {"account_no":1001},
            {"$inc":{"balance":-5000}},
            session=session
        )

        accounts.update_one(
```

```python
            session=session
        )

        if sender["balance"] < 10000:
            raise Exception("Insufficient Balance")

        accounts.update_one(
            {"account_no":1001},
            {"$inc":{"balance":-5000}},
            session=session
        )

        accounts.update_one(
            {"account_no":1002},
            {"$inc":{"balance":5000}},
            session=session
        )

        print("Transaction Successful")

except Exception as e:

    print("Transaction Failed:", e)

finally:

    session.end_session()
```

7. MongoDB handles consistency, isolation, and concurrent access in distributed systems:

7.1Consistency

MongoDB ensures:

- Correct data
- No invalid state
- Proper replica synchronization

7.2Isolation

Transactions are isolated using:

- Snapshot isolation
- Document locking

Other users cannot see incomplete data.

7.3Concurrent Access

MongoDB handles concurrency using:

- Atomic operations
- OCC
- Transactions
- Locking

7.4Distributed System Support

MongoDB supports:

- Replica Sets
- Sharding

- Read Concern
- Write Concern

Benefits:

- High availability
- Scalability
- Fault tolerance

8. Compare normal updates with transaction-based updates:

Feature	Normal update	Transaction
Scope	Single document	Multiple documents
Rollback	No	Yes
Atomicity	Limited	Full
Consistency	Medium	High
Speed	Fast	Slower
Complexity	Easy	Complex

8.1 Advantages Of Normal Update:

- Fast
- Easy
- Low overhead

8.2Limitations:

- No rollback
- Risk of inconsistency

8.3Advantages Of Transactions:

- Strong consistency
- Reliable banking system
- Safe concurrent access

8.4S

- Slightly slower
- More resource usage

9. Role Of Versioning, Atomic Operations, And Transactions:

9.1Versioning

Used in OCC.

Purpose:

- Detect concurrent modifications
- Prevent lost updates

9.2Atomic Operations

MongoDB single document operations are atomic.

Example:

```
> db.accounts.updateOne(
    {account_no:1001},
    {$inc:{balance:1000}}
  )
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
Atlas atlas-i5oclg-shard-0 [primary] BankDB>
```

```
_id: ObjectId('6a09ed9185a3d9ff5fec6a3f')
account_no : 1001
holder_name : "Ali Khan"
balance : 41000
version : 2
```

Safe concurrent update.

9.3Transactions:

Provide:

- Multi-document consistency
- Rollback support
- ACID guarantees

Important in:

- Banking
- E-commerce
- Financial systems

9.4 Observations:

- Concurrent updates may overwrite data
- OCC successfully detects conflicts
- Transactions maintain consistency
- Rollback prevents invalid updates
- MongoDB provides reliable concurrency control

9.5 Challenges Faced:

- Configuring replica set
- Understanding transactions
- Managing concurrent operations
- Handling sessions in Python

10. CONCLUSION

MongoDB provides powerful concurrency control mechanisms including:

- Atomic operations
- Versioning
- Transactions

These features help maintain:

- Consistency
- Reliability
- Isolation
- Fault tolerance

MongoDB transactions are highly useful in enterprise and banking applications where data integrity is critical.